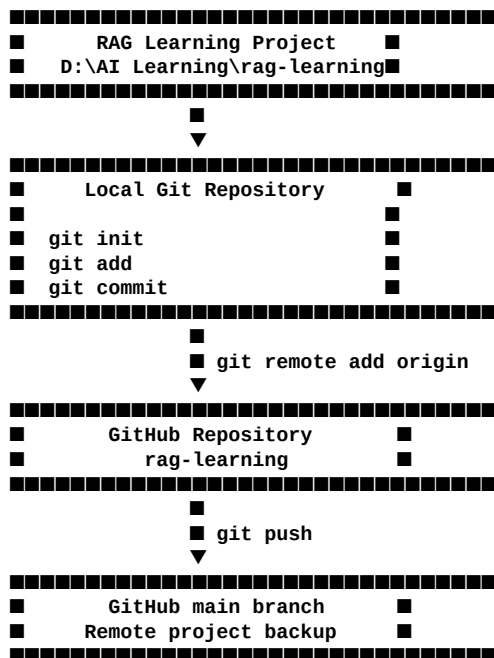


# Git Integration Process — RAG Learning Project

## 1. Objective

We created a dedicated **rag-learning** project and integrated it with Git so that the project can be version-controlled locally and pushed to GitHub.

Overall flow:



## 2. Project root

We worked from the project root:

```
cd "D:\AI Learning\rag-learning"
```

The project contains the RAG learning materials and environment configuration.

## 3. Initialize Git

We initialized Git for the project:

```
git init
```

This creates a hidden **.git** directory and turns the project directory into a local Git repository.



## 4. Check Git status

We checked the current state using:

```
git status
```

This allows us to see which files are untracked, modified, staged, or ready to commit.

## 5. Check .gitignore

Before committing, we made sure files that should not be committed are ignored.

```
.venv/
```

```
__pycache__/  
*.pyc  
.env  
.pytest_cache/  
.ipynb_checkpoints/
```

Important principle:

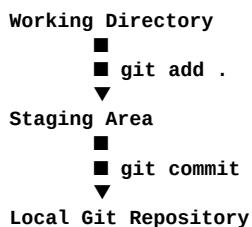
```
COMMIT:  
  Source code  
  Configuration files  
  Documentation  
  uv.lock  
  Project files  
  
DO NOT COMMIT:  
  .venv/  
  .env  
  API keys  
  Secrets  
  Generated cache files
```

## 6. Stage the project

We staged the project files using:

```
git add .
```

The Git flow is:



## 7. Create the first commit

We created the initial commit using:

```
git commit -m "Created a dedicated RAG learning project"
```

The resulting commit was:

```
[master (root-commit) bcdfca8] Created a dedicated RAG learning project  
8 files changed, 443 insertions(+)
```

The committed files included:

```
.gitignore  
.python-version  
README.md  
docs/Environment Setup/RAG_Environment_Setup_uv.pdf  
docs/RAG_Learning_Roadmap.pdf  
main.py  
pyproject.toml  
uv.lock
```

This confirmed that the local Git repository was successfully created and the initial project version was saved.

## 8. Attempt to push

We then tried:

```
git push
```

Git returned:

```
fatal: No configured push destination.  
Either specify the URL from the command-line or configure a remote repository using  
  
git remote add <name> <url>
```

and then push using the remote name

Understanding the error: this was not a problem with the commit. The local repository was working correctly. The problem was that no remote GitHub repository had been configured.



## 9. Create the GitHub repository

The next step is to create a dedicated repository on GitHub.

Recommended repository name:

rag-learning

Recommended description:

RAG learning journey from fundamentals to advanced and production-grade RAG.

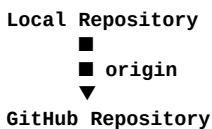
Because the project already exists locally, the GitHub repository should initially be empty. Do not initialize it with a README, .gitignore, or license because these already exist locally.

## 10. Get the GitHub repository URL

After creating the GitHub repository, GitHub provides its repository URL, similar to:

`https://github.com/<YOUR-USERNAME>/rag-learning.git`

This URL is required to connect the local Git repository to GitHub.



## 11. Rename the branch to main

Our initial commit was created on **master**. We decided to use the modern GitHub convention, **main**.

```
git branch -M main
```

Verify:

```
git branch
```

Expected:

```
* main
```

## 12. Add GitHub as the remote

After creating the GitHub repository and copying its URL, connect it to the local repository:

```
git remote add origin https://github.com/<YOUR-USERNAME>/rag-learning.git
```

Here, **origin** is the name/alias of the GitHub remote repository.

## 13. Verify the remote

Always verify the remote before pushing:

```
git remote -v
```

Expected pattern:

```
origin https://github.com/<YOUR-USERNAME>/rag-learning.git (fetch)
origin https://github.com/<YOUR-USERNAME>/rag-learning.git (push)
```

This verification step is especially useful for catching a wrong repository name or wrong GitHub account before pushing.

## 14. Push the project to GitHub

Once the remote is correct and the branch is **main**, push the existing commit:

```
git push -u origin main
```

The **-u** option establishes the upstream relationship between the local **main** branch and **origin/main**.

After the first successful push, future pushes can simply use:

```
git push
```

## 15. Verify the final state

Run:

```
git status
```

The ideal result is:

On branch main

Your branch is up to date with 'origin/main'.

```
nothing to commit, working tree clean
```

Then refresh the GitHub repository page and verify that the project files are visible and that **.venv/** and **.env** are not committed.

## 16. Everyday Git workflow for the RAG journey

As we progress through the RAG stages, the normal workflow will be:

```
Make changes
  ▼
git status
  ▼
git add .
  ▼
git commit -m "meaningful message"
  ▼
git push
  ▼
GitHub
```

## 17. Recommended commit strategy

Use meaningful commits for each major learning milestone:

```
Initial RAG learning environment setup
Stage 1: Implement PDF ingestion
Stage 1: Add DOCX ingestion
Stage 1: Add metadata extraction
Stage 2: Implement document processing
Stage 3: Implement recursive chunking
Stage 4: Add embedding generation
Stage 5: Implement vector indexing
Stage 6: Implement dense retrieval
```

This creates a useful Git history of the RAG learning journey instead of vague commit messages such as "update", "changes", or "final".

## 18. Final Git + GitHub architecture

```
RAG LEARNING PROJECT
  ▼
Local Working Directory
  ▼
git add .
  ▼
```

